

Chapter 6: CPU Scheduling

Shatabdi Roy Moon
Lecturer, Dept. of CSE



Chapter 6: CPU Scheduling

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Examples

Objectives

- To introduce CPU scheduling, which is the basis for multi-programmed operating systems
- To describe various CPU-scheduling algorithms
- To discuss evaluation criteria for selecting a CPU-scheduling algorithm for a particular system

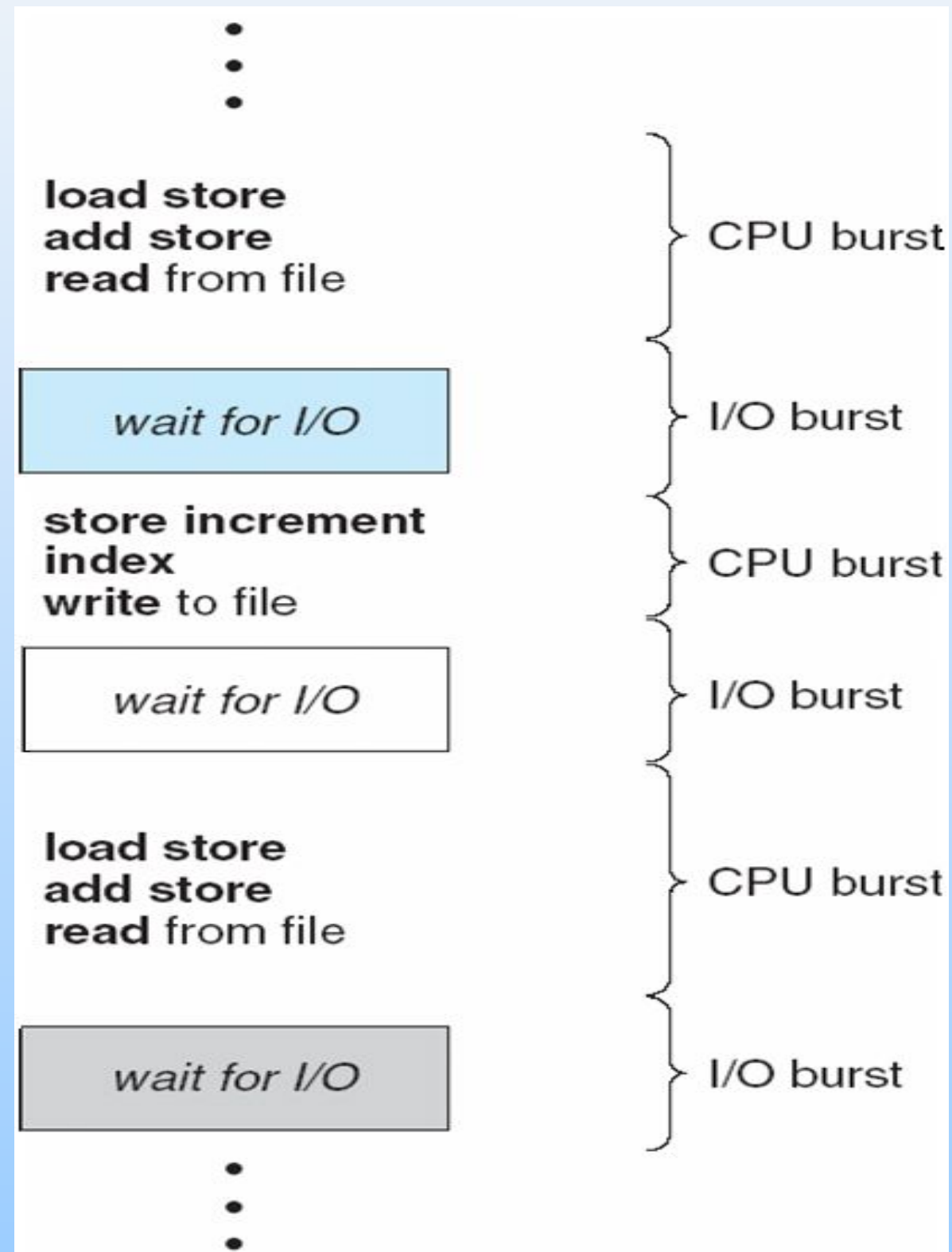
Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- Continuous Cycle :
 - one process has to wait (I/O)
 - Operating system takes the CPU away
 - Give CPU to another process
 - This pattern continues
- CPU–I/O Burst Cycle – Process execution consists of a *cycle* of CPU execution and I/O wait

CPU and I/O Burst Cycle

- Almost all processes alternate between two states in a continuing *cycle*, as shown in Figure below :
 - A CPU burst of performing calculations, and
 - An I/O burst, waiting for data transfer in or out of the system.
- Processes alternate back and forth between these two states.

Alternating Sequence of CPU and I/O Bursts



CPU Scheduler

- Whenever the CPU becomes idle, it is the job of the CPU Scheduler (a.k.a. the short term scheduler) to select another process from the ready queue to run next.
- The storage structure for the ready queue and the algorithm used to select the next process are not necessarily a FIFO queue. There are several alternatives to choose from, as well as numerous adjustable parameters for each algorithm.
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state (I/O request)
 2. Switches from running to ready state (e.g. when interrupt occurs)
 3. Switches from waiting to ready (e.g. at completion of I/O)
 4. Terminates
- Scheduling under 1 and 4 is **non-preemptive**
- All other scheduling is **preemptive**
 - Consider access to shared data
 - Consider preemption while in kernel mode
 - Consider interrupts occurring during crucial OS activities

Dispatcher

- The dispatcher is the module that gives control of the CPU to the process selected by the scheduler. This function involves:
 - Switching context.
 - Switching to user mode.
 - Jumping to the proper location in the newly loaded program.
- The dispatcher needs to be as fast as possible, as it is run on every context switch. The time consumed by the dispatcher is known as **dispatch latency**.

Scheduling Criteria

There are several different criteria to consider when trying to select the "best" scheduling algorithm for a particular situation and environment, including:

- **CPU utilization** – Ideally the CPU would be busy 100% of the time, so as to waste 0 CPU cycles. On a real system CPU usage should range from 40% (lightly loaded) to 90% (heavily loaded.)
- **Throughput** – Number of processes completed per unit time. May range from 10/second to 1/hour depending on the specific processes.
- **Turnaround time**
 - amount of time to execute a particular process
 - the interval from the time of submission of a process to the time of the completion.
- **Waiting time** – Amount of time a process has been waiting in the ready queue.
(**Load average** – The average number of processes sitting in the ready queue waiting their turn to get into the CPU)
- **Response time** – Amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

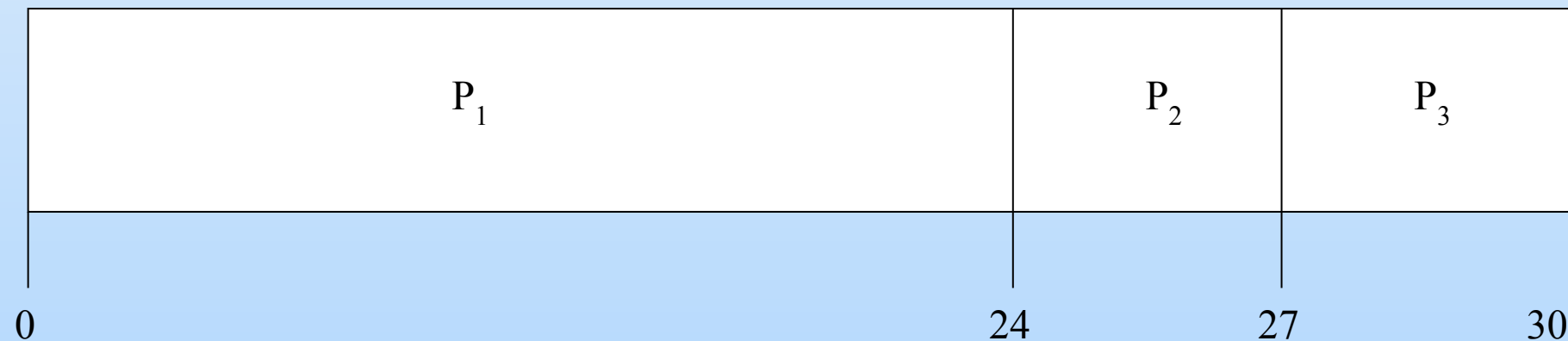
Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

First-Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$
- Turnaround time $P_1 = 24$; $P_2 = 27$; $P_3 = 30$

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:



Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$

Average waiting time: $(6 + 0 + 3)/3 = 3$

Much better than previous case

Convoy effect - short process behind long process Consider one CPU-bound and many I/O-bound processes

FCFS Scheduling (Cont.)

Convoy Effect :

many I/O bound process and one CPU bound process

CPU bound process	I/O bound process	Effect
I/O device	I/O queue	CPU site idle
CPU processing	Ready queue	I/O site idle

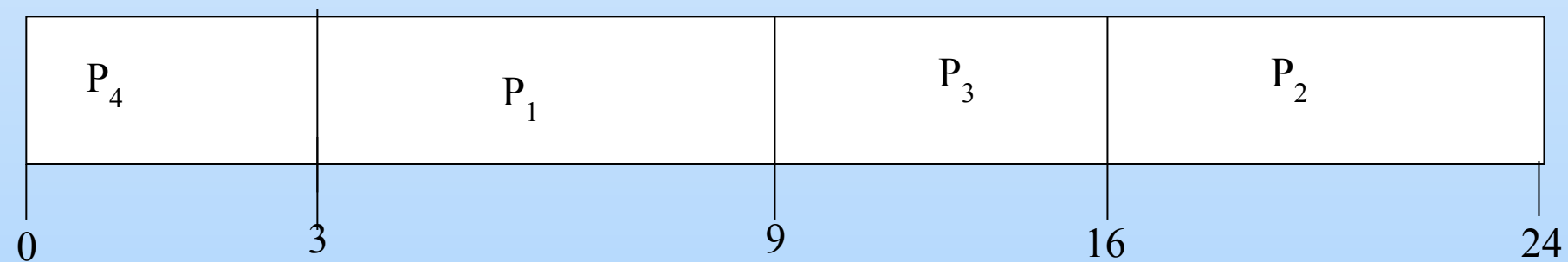
Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time
- Two schemes:
 - **Non-preemptive** – Once CPU given to the process it cannot be preempted until completes its CPU burst
 - **Preemptive** – If a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is known as the **Shortest-Remaining-Time-First (SRTF)**
- SJF is optimal – gives minimum average waiting time for a given set of processes

Example of SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	6
P_2	2.0	8
P_3	4.0	7
P_4	5.0	3

- SJF scheduling chart

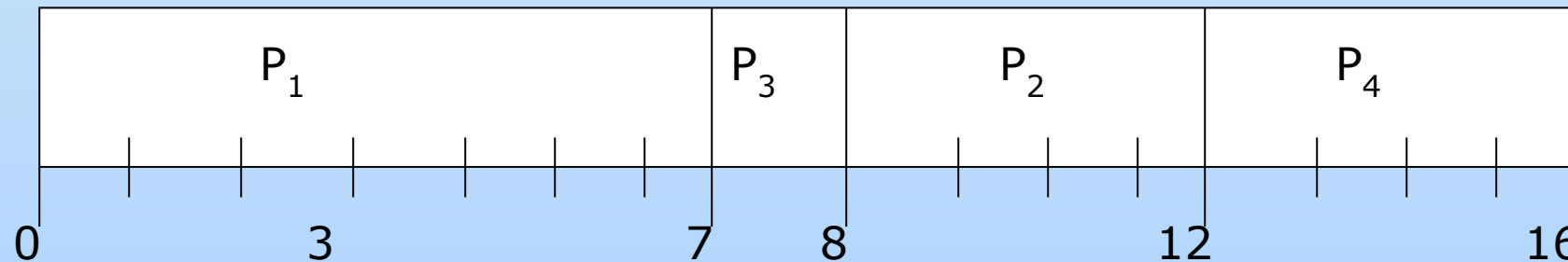


$$\text{Average waiting time} = (3 + 16 + 9 + 0) / 4 = 7$$

Example of Non-Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)

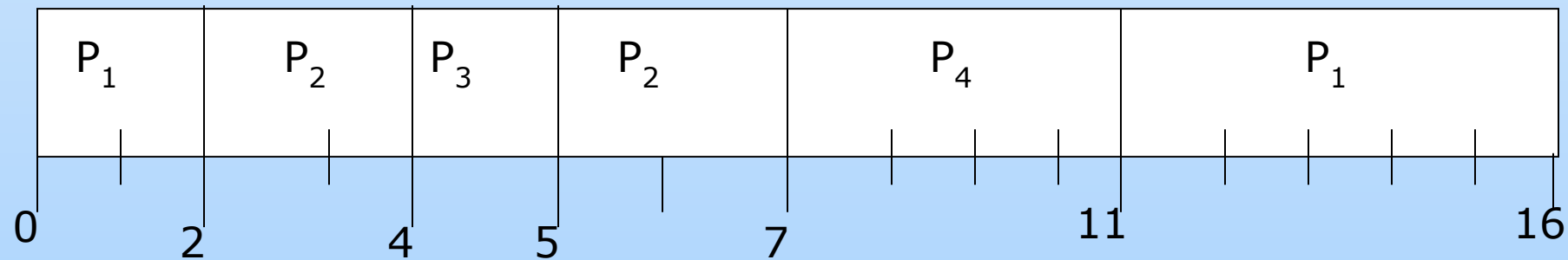


$$\text{Average waiting time} = (0 + 6 + 3 + 7)/4 = 4$$

Example of Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



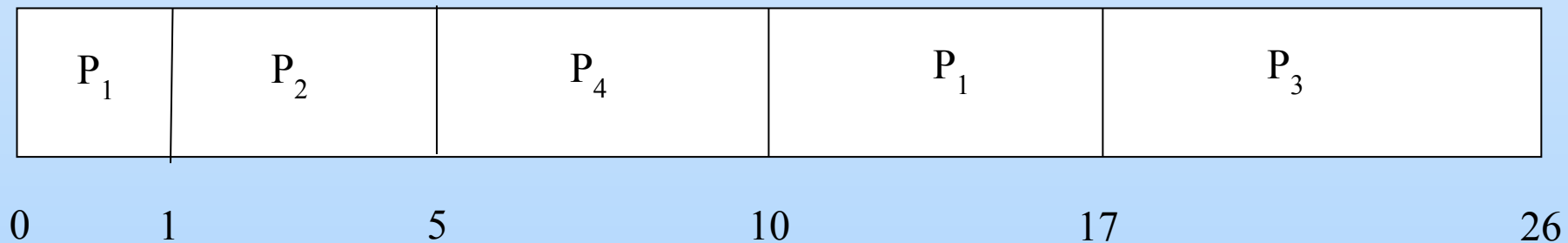
- Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

Example of Shortest-remaining-time-first

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- Preemptive* SJF Gantt Chart



- Average waiting time = $[9+0+15+2]/4 = 28/4 = 6.5\text{ms}$

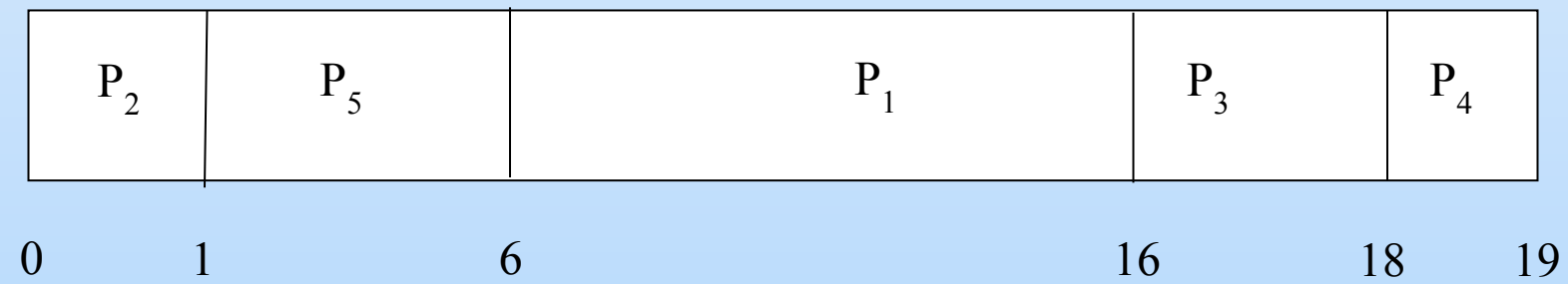
Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - Non-preemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Priority can be defined either internally or externally.
 - Factors for internal priority assignment:
 - 4 Time limit, memory requirements, the number or open files etc.
 - Factors for external priority assignment:
 - 4 Importance of the process, the type and amount of funds of funds being paid for computer use, department sponsoring works etc.

Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart



Average waiting time = 8.2 msec

Priority Scheduling

- Problem $\equiv$ **Starvation** – low priority processes may never execute
- Solution $\equiv$ **Aging** – as time progresses increase the priority of the process

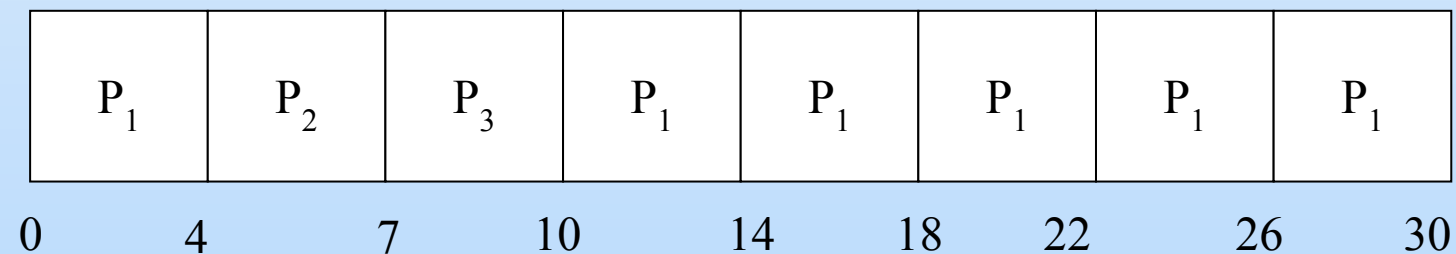
Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum** q), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

Example of RR with Time Quantum = 4

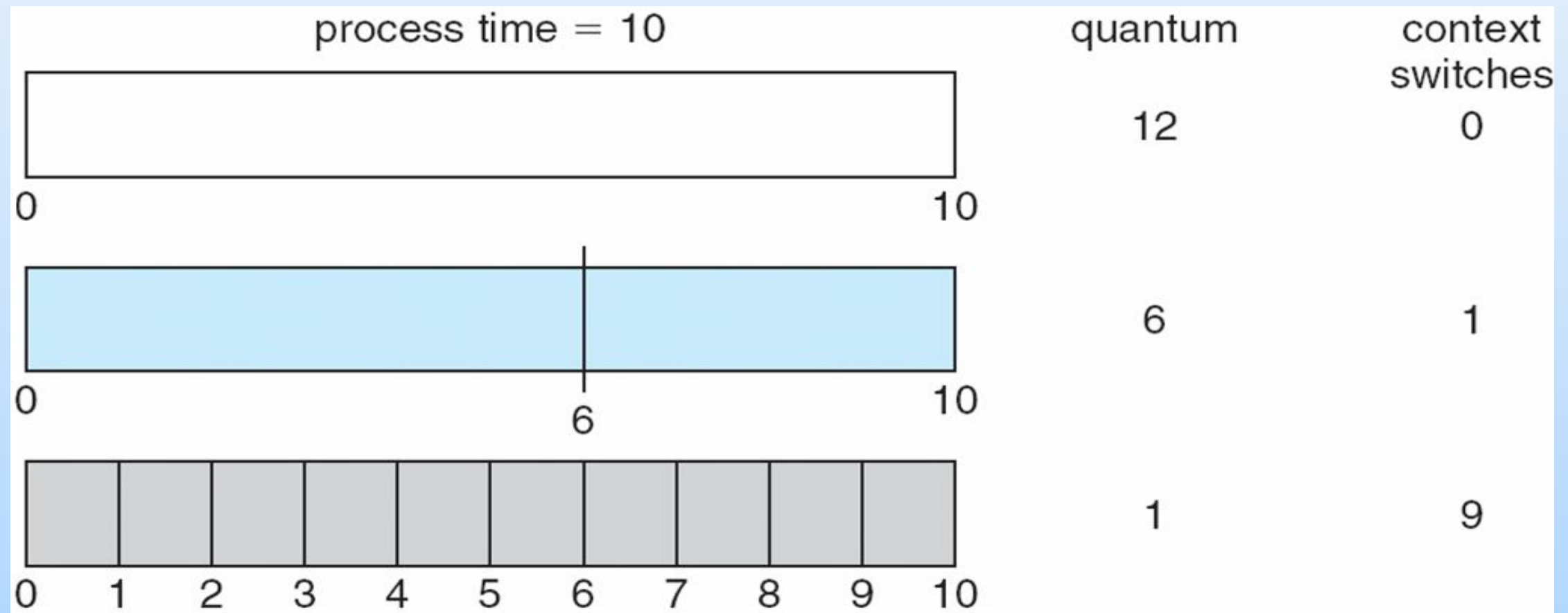
<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The Gantt chart is:

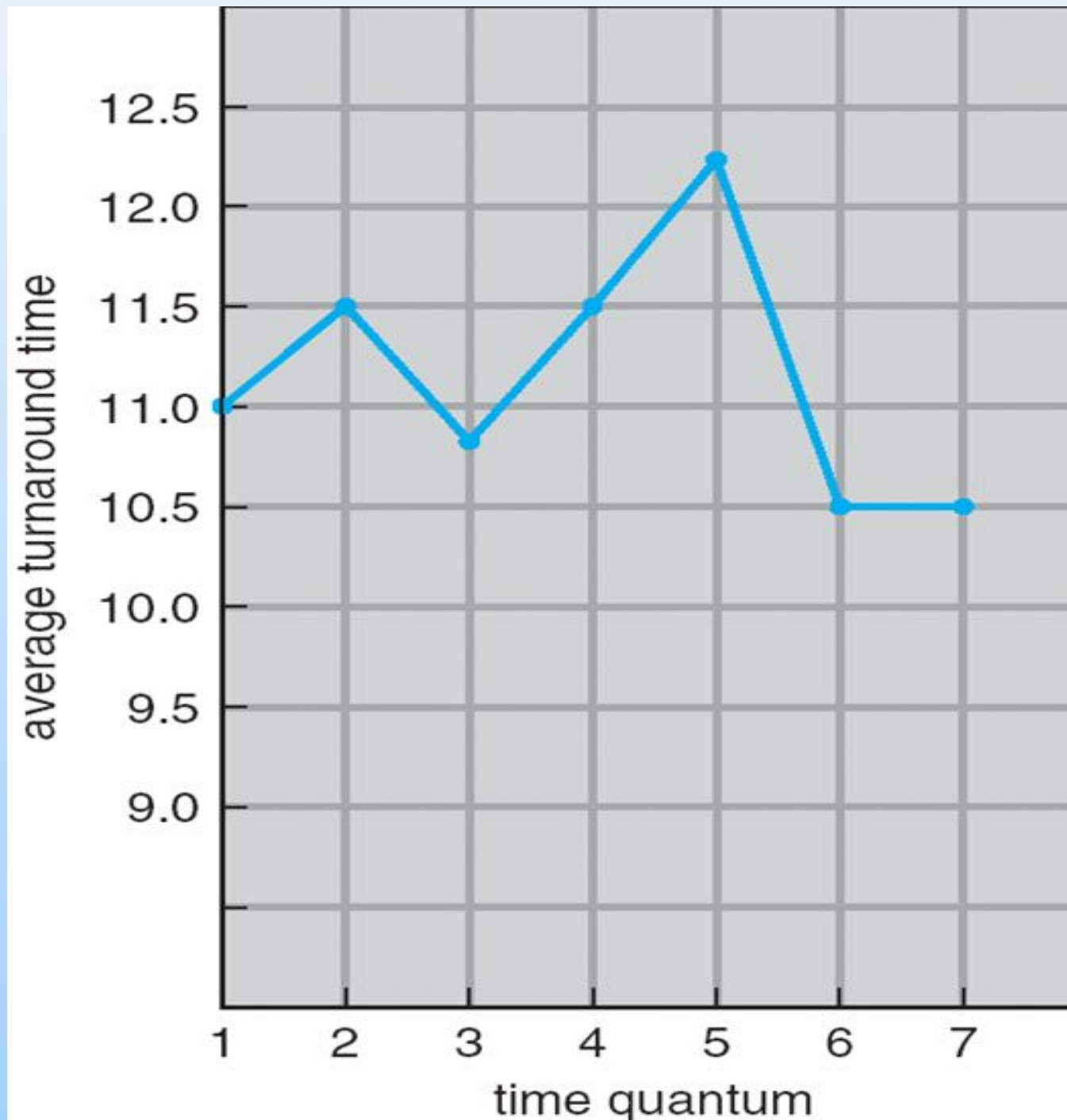


- Average waiting time is $17 / 3 = 5.66$ millisecond
- Typically, higher average turnaround than SJF, but better *response*
- quantum should be large compared to context switch time
- Quantum usually 10ms to 100ms, context switch < 10

Time Quantum and Context Switch Time



Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts should be shorter than quantum

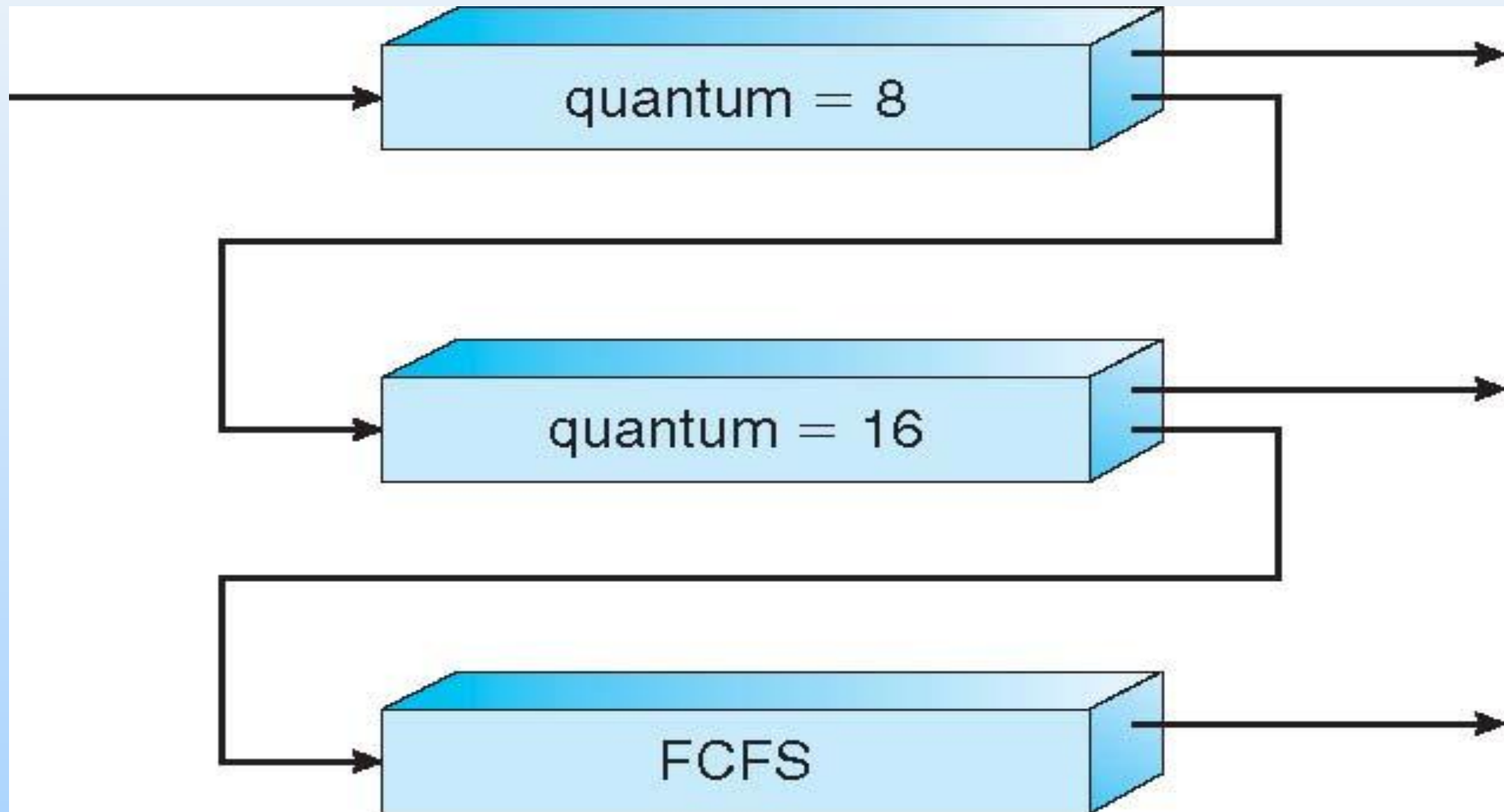
Multilevel Queue

- Another class of scheduling algorithm needs- in which processes are classified into different groups, e.g.:
 - foreground (interactive) processes
 - background (batch) processes
- They have different response time requirements-so different scheduling needs.
- Foreground processes may have priority over background processes.
- A multilevel queue-scheduling algorithm partitions the ready queue into several separate queues-we can see it in the figure of next slide:-
- Each queue has its own scheduling algorithm:
 - Foreground queue scheduled by – RR algorithm
 - Background queue scheduled by – FCFS algorithm
- Scheduling must be done between the queues:
 - Fixed priority preemptive scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., foreground queue can be given 80% of the CPU time for RR-scheduling among its processes, while 20% to background in FCFS manner.

Example of Multilevel Feedback Queue

- Three queues: (can see the figure in next slide)
 - Q_0 – RR with time quantum 8 milliseconds
 - Q_1 – RR time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new job enters queue Q_0 which is served for RR
 - 4 When it gains CPU, job receives 8 milliseconds
 - 4 If it does not finish in 8 milliseconds, job is moved to queue Q_1
 - At Q_1 job is again served RR and receives 16 additional milliseconds
 - 4 If it still does not complete, it is preempted and moved to queue Q_2

Multilevel Feedback Queues



End of Chapter 6

